

UNIVERSITY OF KABIANGA

DEPARTMENT OF COMPUTER SCIENCE

COURSE: BCS 404 – SECURE SOFTWARE ENGINEERING

Name :BURTONE HEEDS OPENO

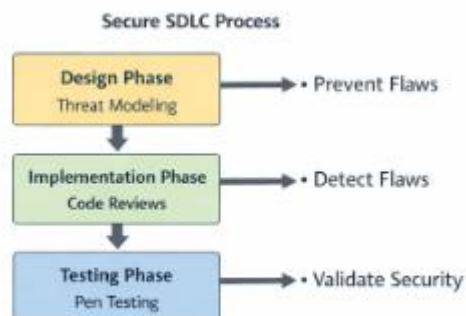
REG: COM/M/0047/2022

TAKE AWAY ASSINGMENT

SECTION 1: Secure SDLC and Threat Modeling (15 Marks)

Scenario: You are the lead security architect for a FinTech startup building a new mobile banking application.

1. **SDLC Integration (7 Marks):** Propose a strategy for integrating security into the following three phases of the SDLC. For each phase, describe one specific security activity and explain its contribution to the overall security posture:
 - **Design Phase:** (e.g., Attack Surface Analysis or Threat Modeling).
 - **Implementation Phase:** (e.g., Secure Code Reviews or IDE-based security plugins).
 - **Testing Phase:** (e.g., DAST or Penetration Testing).



Dagram 1.1

ANSWER

Design Phase – Threat Modeling

What: Structured identification of threats using STRIDE.

Why:

Security flaws introduced at the design stage propagate through the system and become costly to fix later. Early threat modeling reduces systemic vulnerabilities (Sigman, 2016).

How:

- Identify assets (e.g., user funds, credentials)
- Model attacker capabilities
- Apply STRIDE categories

Contribution:

Builds **security-by-design**, preventing architectural weaknesses before implementation.

Implementation Phase – Secure Code Reviews

What: Reviewing code manually and with automated tools (SAST).

Why:

Developers often introduce vulnerabilities such as improper input validation. Code reviews detect these issues before deployment (Seacord, 2013).

How:

- Use static analysis tools
- Conduct peer reviews focusing on:
 - Input validation
 - Authentication logic
 - API usage

Contribution:

Reduces vulnerabilities at the **code level**, strengthening the application's attack resistance.

Testing Phase – Penetration Testing / DAST

What: Simulated attacks on the running system.

Why:

Certain vulnerabilities (e.g., session flaws, misconfigurations) only appear during runtime (Warren, 2014).

How:

- Use tools like OWASP ZAP
- Perform real-world attack simulations

Contribution:

Validates the system against **realistic threats**, ensuring operational security.

2.Threat Modeling (8 Marks): Using the **STRIDE** methodology, analyze the "Funds Transfer" feature. Construct a table containing **four** of the STRIDE categories (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege). For each selected category, identify a credible threat and propose one specific technical mitigation.

ANSWER

STRIDE Category	Threat	Mitigation	Explanation (Why & How)
Spoofing	Attacker impersonates a user	Multi-Factor Authentication (MFA)	Why: Passwords alone are insufficient. How: MFA adds an additional factor (OTP/biometric), reducing identity theft risk (Seacord, 2013).
Tampering	Modification of transaction data	TLS + Integrity Checks	Why: Data must not be altered in transit. How: Encryption and hashing ensure confidentiality and integrity (Warren, 2014).
Repudiation	User denies transaction	Digital signatures + audit logs	Why: Systems must prove actions occurred. How: Cryptographic signatures and logs create verifiable evidence.
Information Disclosure	Exposure of sensitive data	Encryption + access control	Why: Financial data is highly sensitive. How: Encrypt data and restrict access using least privilege (Sigman, 2016).

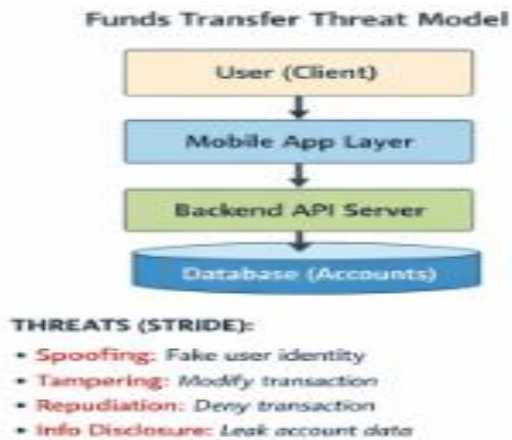


Diagram 1.2

SECTION 2: Design Principles and Security Patterns (12 Marks)

1. Design Principles (6 Marks):

- Explain the principle of "**Economy of Mechanism**" and "**Fail-safe Defaults.**"
- For each, provide a real-world software design scenario where violating these principles leads to a vulnerability (e.g., an overly complex authentication module or a system that defaults to "allow all").

ANSWER

Economy of Mechanism

Definition: Security mechanisms should be simple and minimal.

Why:

Complex systems increase the likelihood of hidden vulnerabilities due to difficulty in analysis (Warren, 2014).

Violation Example:

A complex authentication system with multiple conditional paths.

How it leads to vulnerability:

Developers overlook edge cases → attackers exploit logic flaws.

Fail-safe Defaults

Definition: Systems should deny access by default.

Why:

Default “allow” policies expose systems when rules are missing (Seacord, 2013).

Violation Example:

An API that allows access unless explicitly blocked.

How it leads to vulnerability:

Missing restrictions → unauthorized access.

2. Security Patterns (6 Marks):

- Analyze how the **Role-Based Access Control (RBAC)** pattern manages authorization complexity compared to individual user permissions.
- Explain how the "**Check-point**" pattern functions as a gatekeeper in a multi-step workflow (e.g., a multi-factor authentication flow or a multi-stage payment gateway).

ANSWER**Role-Based Access Control (RBAC)**

What: Permissions are assigned to roles, not individuals.

Why:

Managing individual permissions is complex and error-prone (Sigman, 2016).

How it works:

- Assign roles (Admin, User)
- Assign permissions to roles

Security Benefit:

Reduces misconfiguration and enforces **least privilege**.

Check-point Pattern

What: A control point that verifies conditions before progression.

Example: Multi-Factor Authentication

How:

1. User logs in
2. System pauses (checkpoint)
3. Requires OTP
4. Grants access if valid



Diagram 2.1

Why:

Prevents attackers from progressing without verification (Warren, 2014).

SECTION 3: Secure Coding and Dependency Management (18 Marks)

1. Practical Vulnerability Mitigation (10 Marks):

- **The Flaw:** Write a code snippet (e.g., in Java, PHP, or Python) vulnerable to **SQL Injection** that concatenates user input directly into a query. (3 Marks)
- **The Fix:** Rewrite the code using **Parameterized Queries (Prepared Statements)**. (4 Marks)
- **The Logic:** Explain the technical difference between the two—specifically, how the secure version separates "data" from "code" to neutralize the attack. (3 Marks)

ANSWER

Vulnerable Code

pyhon

```
username = input("Enter username: ")  
query = "SELECT * FROM users WHERE username = '" + username + "';"  
cursor.execute(query)
```

Secure Code (Parameterized Query)

pyhon

```
username = input("Enter username: ")  
query = "SELECT * FROM users WHERE username = %s;"  
cursor.execute(query, (username,))
```

The Logic (Why & How)

Vulnerable Version:

- Combines user input with SQL query
- Input becomes executable code

Attack Example:

```
' OR '1'='1
```

Secure Version:

- Separates SQL logic from user input

How it works:

The database treats input strictly as **data**, not code.

Why effective:

Prevents execution of malicious input (Seacord, 2013).

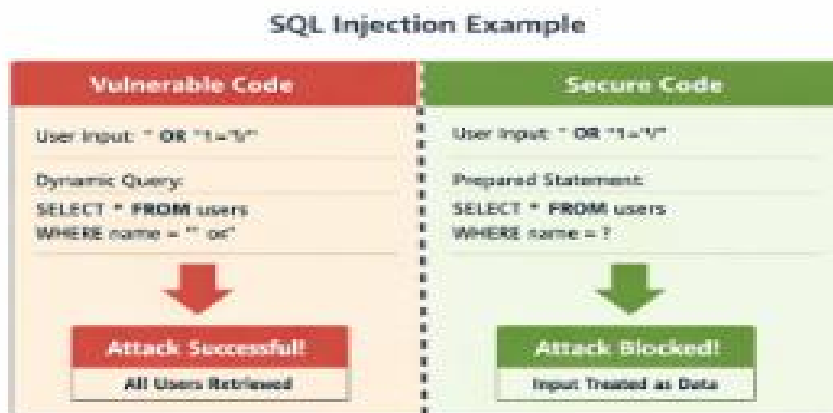


Diagram 3.1

2. Software Composition Analysis - SCA (8 Marks):

- Define SCA and discuss why it is critical given the reliance on open-source libraries in modern software development. **(4 Marks)**
- Outline the step-by-step lifecycle of managing third-party dependencies: **Identification** (Software Bill of Materials), **Assessment** (Vulnerability Scanning), and **Remediation** (Patching/Upgrading). **(4 Marks)**

ANSWER

Definition

SCA identifies and manages risks in third-party libraries.

Why It Is Critical

Modern applications depend heavily on open-source components.

Risk:

Vulnerabilities in dependencies compromise the system (Warren, 2014).

Lifecycle

1. Identification (SBOM)

What: List all dependencies

Why:

Unknown components = unmanaged risk (Sigman, 2016).

2. Assessment (Vulnerability Scanning)

What: Scan for known vulnerabilities

Why:

Detect known threats early

3. Remediation (Patching)

What: Update or replace vulnerable components

Why:

Fix known security flaws

SECTION 4: Verification, Quality Assurance, and Compliance (15 Marks)

1. Benchmarking & Metrics (6 Marks):

- Define **Security Benchmarking** in the context of software quality.
- Identify and define **three** quantitative metrics (e.g., *Vulnerability Density*, *Mean Time to Remediate (MTTR)*, or *Code Coverage of Security Tests*) that a manager would use to assess the team's security performance.

ANSWER

Security Benchmarking

Definition

Measuring security performance using metrics.

Why:

Provides objective evaluation of security posture (Sigman, 2016).

Metrics

1. Vulnerability Density

Number of vulnerabilities per LOC

Why:

Indicates code quality

2. Mean Time to Remediate (MTTR)

Time to fix vulnerabilities

Why:

Faster remediation reduces exposure (Seacord, 2013).

3. Security Test Coverage

Percentage of code tested

Why:

Higher coverage → fewer undetected vulnerabilities

2. Regulatory Impact (6 Marks):

- Select **GDPR or PCI DSS**.
- Explain how this regulation mandates specific security requirements during the **Requirements Phase**. Give examples of how this affects data storage, encryption, or user consent documentation.

ANSWER

Why GDPR Matters

Protects personal data and enforces accountability.

Requirements Phase Impact

Data Minimization

Collect only necessary data

Why:

Reduces breach impact

Encryption

Protect data in transit and at rest

How:

- TLS
- AES

Why:

Prevents unauthorized access (Warren, 2014).

User Consent

Explicit consent must be obtained

Why:

Ensures transparency and compliance

3. Secrets Management (3 Marks):

- Why is "Hardcoding Secrets" (e.g., API keys, database credentials) considered a major security anti-pattern? Briefly describe how using a **Secrets Vault** (e.g., HashiCorp Vault, AWS Secrets Manager) mitigates this risk.

ANSWER

Hardcoding Secrets

python

```
API_KEY = "SECRET123"
```

Why dangerous:

- Easily exposed in repositories
- Difficult to rotate

Secrets Vault Solution

Examples:

- HashiCorp Vault
- AWS Secrets Manager

How it works

- Secrets stored securely outside code
- Access controlled dynamically

Why effective

- Reduces exposure risk
- Enables secure rotation (Seacord, 2013)

• REFERENCES

- Seacord, R. C. (2013). *Secure coding in C and C++* (2nd ed.). Addison-Wesley.
- Warren, R. (2014). *Security engineering: A guide to building dependable distributed systems*. Wiley.
- Sigman, G. (2016). *Building secure software: How to avoid security problems the right way*. Addison-Wesley.

BIBLIOGRAPHY

Anderson, R. (2014). Security engineering: A guide to building dependable distributed systems (2nd ed.). Wiley.

McGraw, G. (2006). Software security: Building security in. Addison-Wesley.

Seacord, R. C. (2013). Secure coding in C and C++ (2nd ed.). Addison-Wesley.